Introduction to Data Modelling

Agenda

- **What is a Data Model?**
- **Types of Data Models**
- **Relational Data Model**
- **Example - Relational Data Model**
- **Database Schema**
- **Keys**
- **Example – Relational Schema with Keys**

What is Data Model?

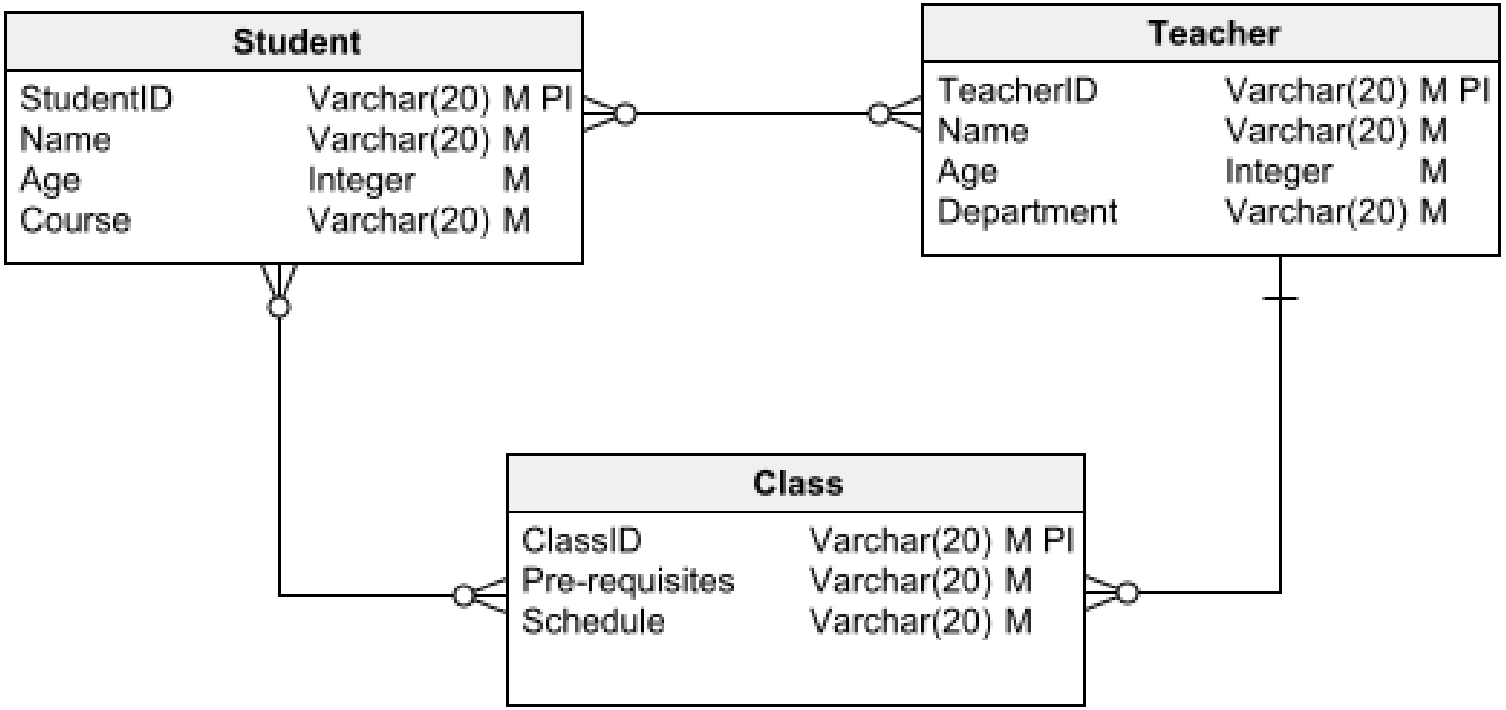
What is Data Model?

- When we move from the **real world** to a **database**, we need a structured way to represent complex, real-life information.
- Data modeling transforms messy, complex real-world data into a structured, logical format suitable for databases.
- A **data model** is a conceptual framework that describes how data is organized, represented, and related to each other in a system.
- In simpler words, it's a **blueprint for structuring data—**it defines what kind of data can be stored, how different pieces of data are connected, and the rules for manipulating it.

Example

- **Scenario:** A university needs to manage student information.
- **Real World:** Students, Courses, Instructors, Enrollments.
- **Data Model:**
  - **Entities:** Student, Course, Instructor
  - **Relationships:** Student *enrolls in* Course; Instructor *teaches* Course
  - **Database Schema:** Tables for Students, Courses, Instructors, and Enrollments

Data Model



Vertabelo

Benefits

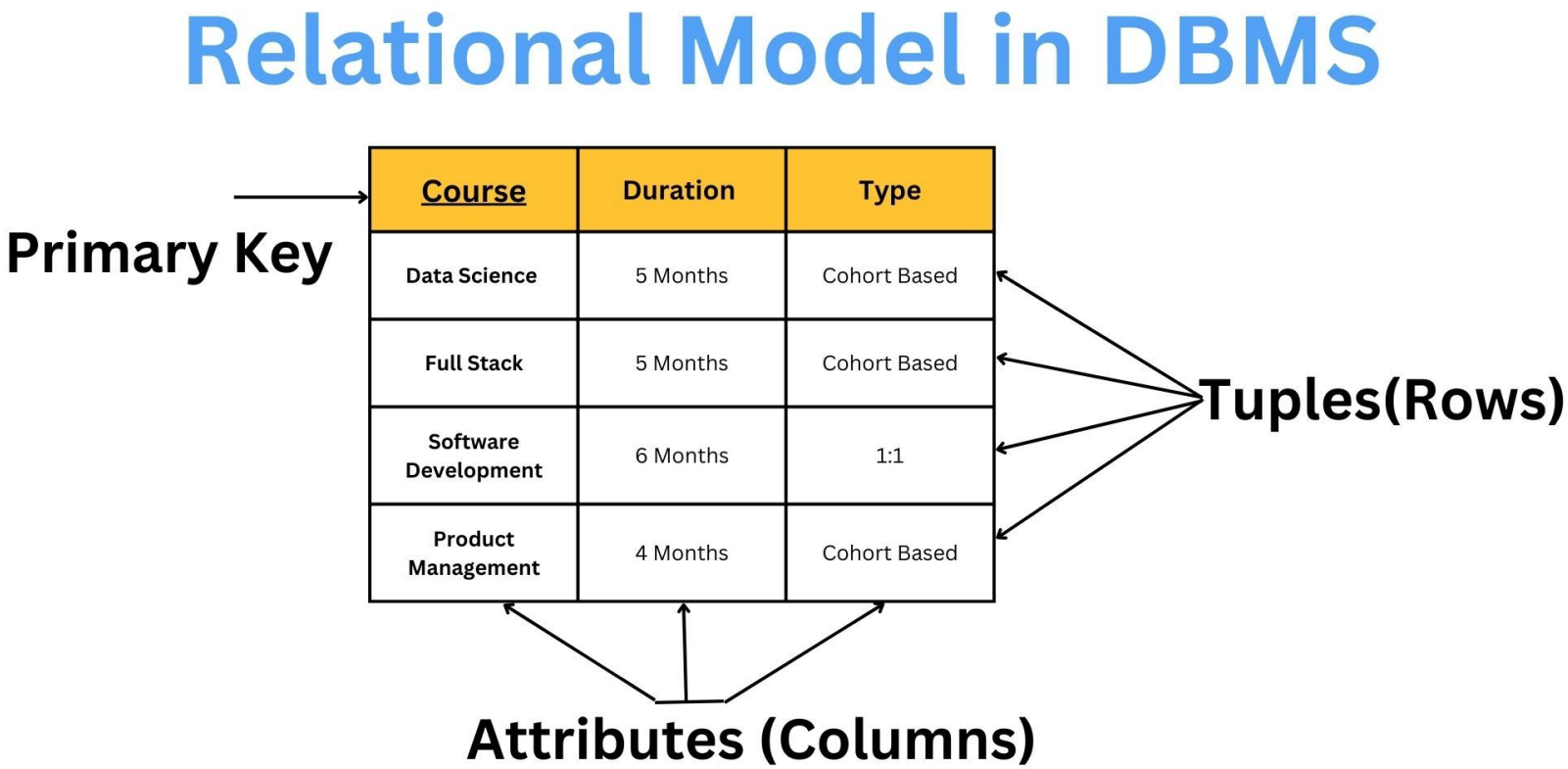
- **Clarity –** Translates business requirements into precise data structures.
- **Consistency –** Reduces redundancy and inconsistency in data.
- **Communication –** Provides a common language between business users and developers.
- **Efficiency –** Leads to better database design, storage, and query performance.
- **Scalability & Maintenance –** Makes future changes easier.

Types of Data Models

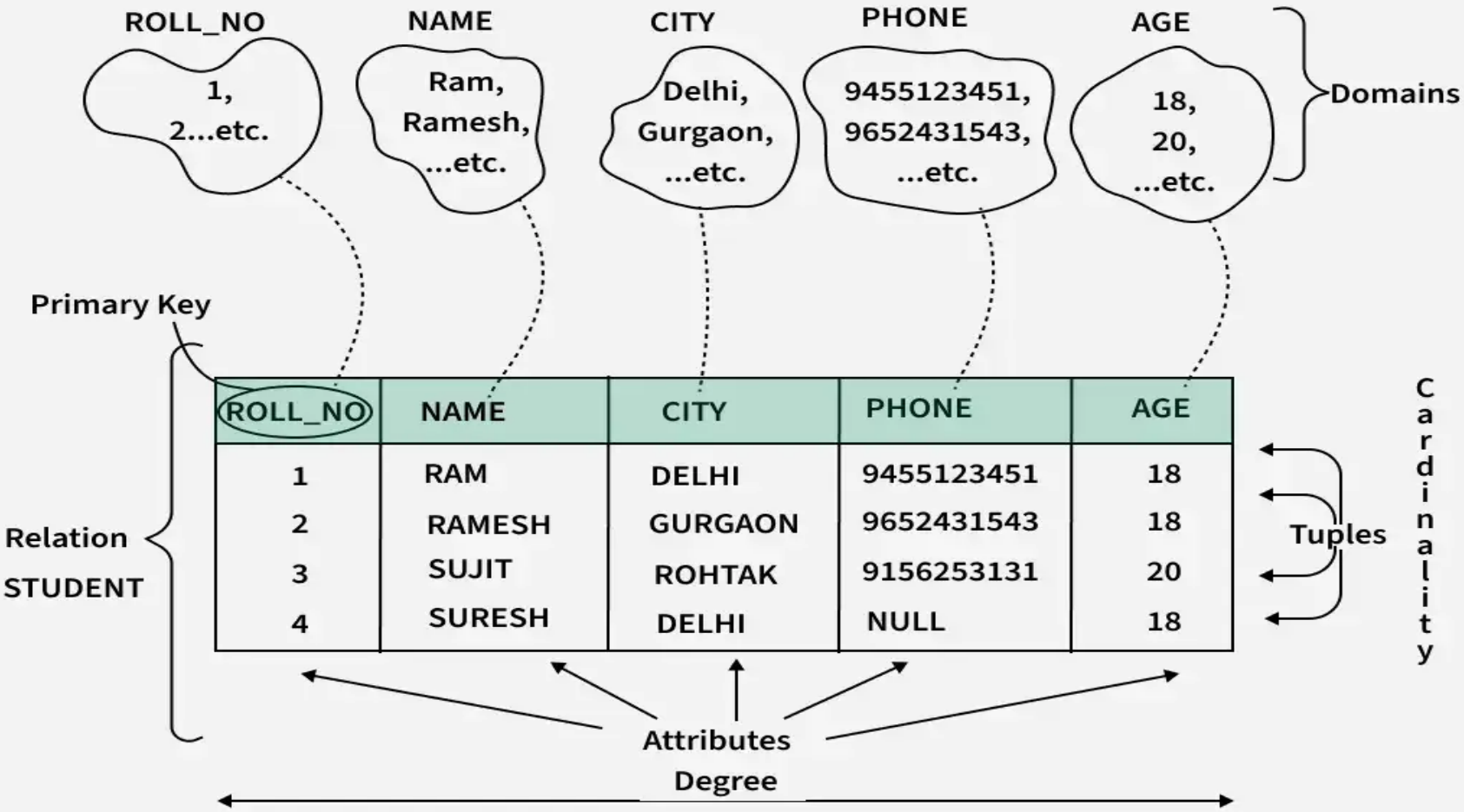
Relational Model

- **Uses a collection of tables to represent both data and their relationships**
- **Tables are also known as relations**
- **Based on record-based model**
- **Each table has multiple columns, and each column must have a unique name**
- **It has to be unique for that particular relation only, and not for the entire database**
- **Each row is called a tuple or record, which represents a particular entity or object**
- **Each record has a fixed number of fields or attributes, which are represented by the columns**
- **Most widely used data model**

Relational Model



Relational Model



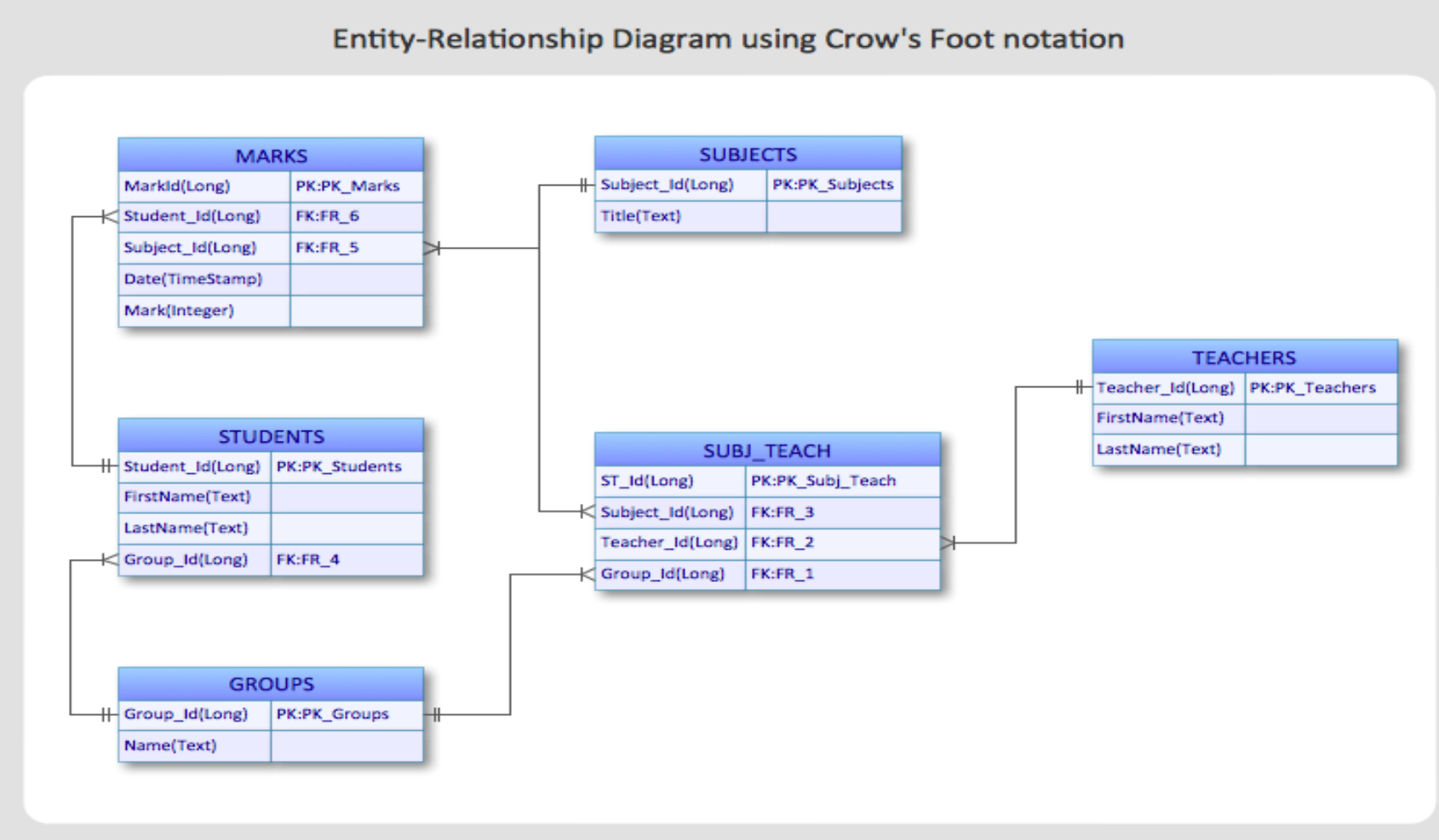
Entity-Relationship Model

- The **Entity–Relationship Model (ER Model)** is one of the most widely used **conceptual data models** for designing databases. It was introduced by Peter Chen in 1976.
- It provides a **high-level, graphical way** of representing the data and its relationships before converting it into tables in a relational database.

Entity-Relationship Model

- Uses a collection of basic objects called ***entities*** and their ***relationships*** among each other
- An ***entity*** can represent any thing or object of the real world
- Depicted by E-R Diagrams

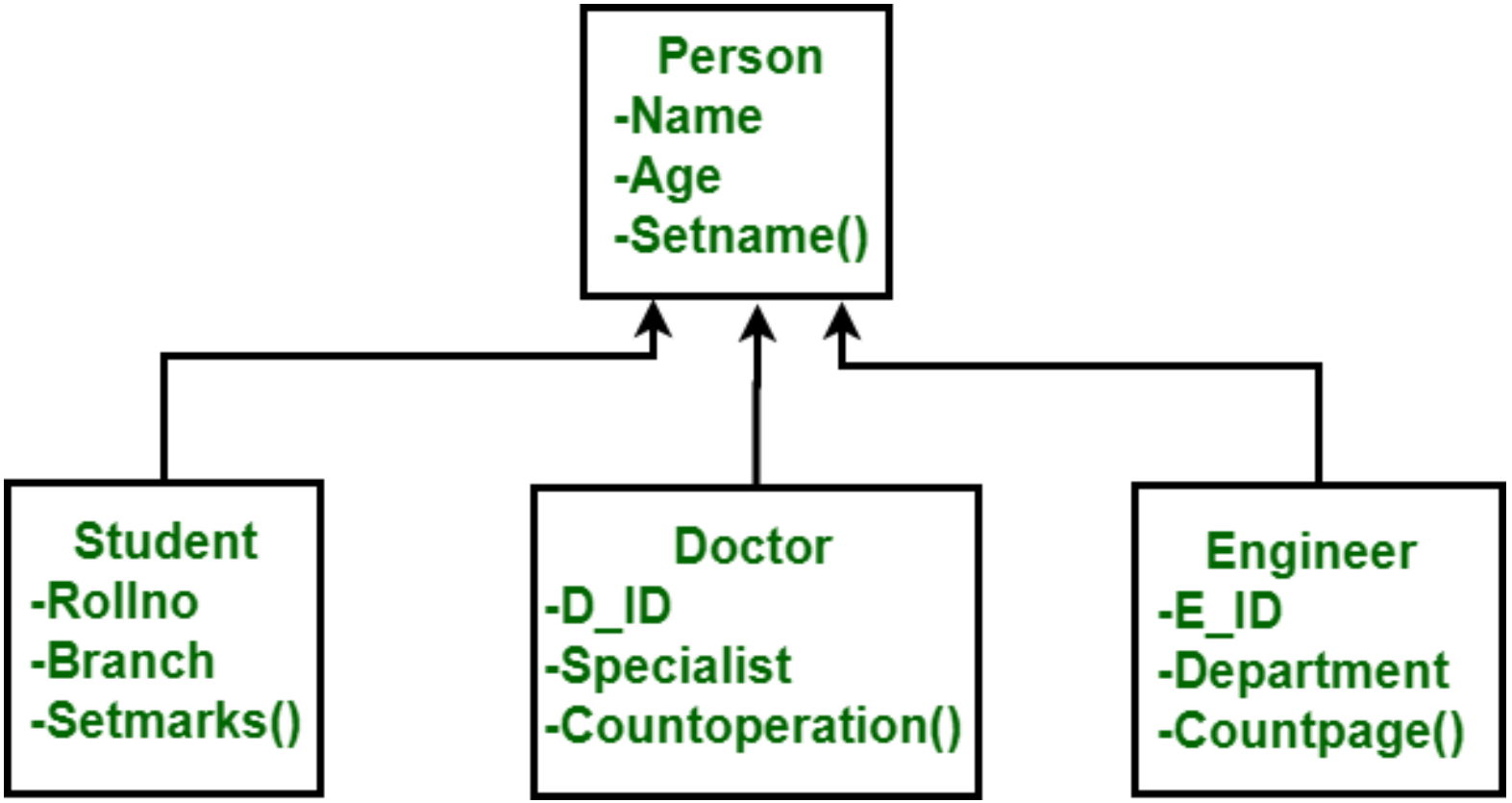
Entity-Relationship Model



Object-Based Model

- An **Object-Based Data Model** is a **high-level conceptual model** used to describe data in terms of **objects**, their **attributes**, and **relationships**. It focuses on representing the real-world entities more **naturally** (like objects in programming) compared to traditional relational models.
- Integrates database concepts with object-oriented programming ideas.
- Supports classes, inheritance, encapsulation, methods.
- Example: Student object can have attributes (Name, Age) and methods (calculateGPA()).

Object-Based Model



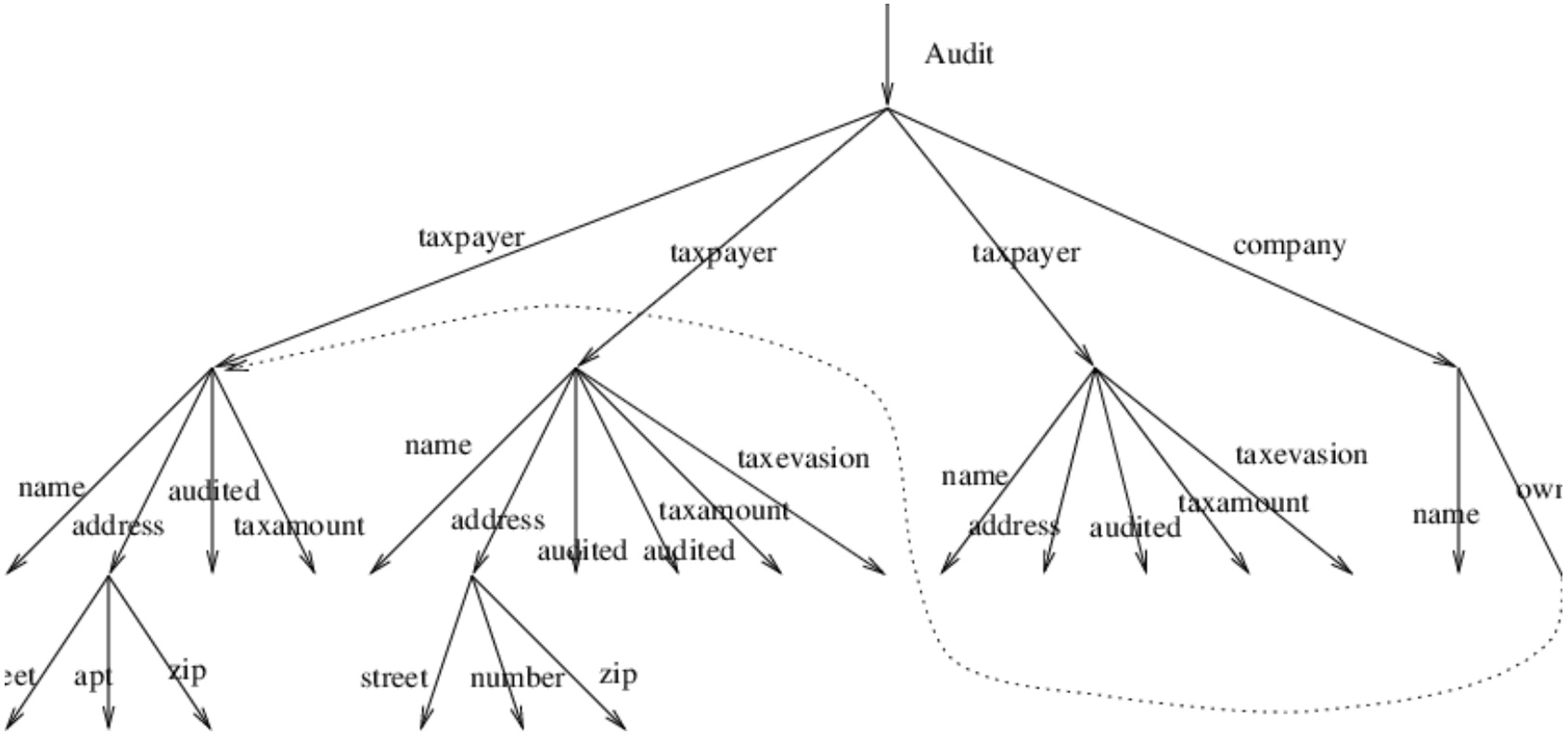
Semi-Structured Model

- A **Semi-structured Data Model** is used to represent data that **doesn’t follow a rigid schema** like relational databases. Instead, it allows **flexibility in structure**, meaning the same type of data may have different attributes, and not all records need to follow the same format.

Semi-Structured Model

- Flexible Schema
- No fixed schema is required in advance.
- Data items may have different sets of attributes.
- Self-Describing
- Data carries structural information along with it (tags, keys, hierarchy).
- Hierarchical / Graph-Based Representation
- Often modeled as tree or graph structures.
- Commonly Used for Heterogeneous Data
- Useful when data comes from different sources with varying formats.

Semi-Structured Model



Relational Data Model

Relational Data Model

The **Relational Data Model (RDM)** is one of the most widely used data models in database systems. It was introduced by **E. F. Codd in 1970** and provides a simple, logical way to represent and manipulate data.

Relational Model

- **Uses a collection of tables to represent both data and their relationships**
- **Tables are also known as relations**
- **Based on record-based model**
- **Each table has multiple columns, and each column must have a unique name**
- **It has to be unique for that particular relation only, and not for the entire database**
- **Each row is called a tuple or record, which represents a particular entity or object**
- **Each record has a fixed number of fields or attributes, which are represented by the columns**
- **Most widely used data model**

Tuple

- **Each row in a table records information, and represents a relationship among a set of values within that table**
- **A single row is technically known as tuple or record**
- **Each attribute in a tuple is related to each other**
- **For example, in the given Instructor table, each row or tuple is representing information about any one specific instructor; and all the attributes are related to each other in such a way that they represent one particular feature of that instructor, which collectively will represent one instructor's information**

Tuple

- **For example, in the given Instructor table, each row or tuple is representing information about any one specific instructor; and all the attributes are related to each other in such a way that they represent one particular feature of that instructor, which collectively will represent one instructor's information**

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Tuple

Example

Consider the relation (table):

STUDENT (StudentID, Name, Program, Age)

StudentID	Name	Program	Age
101	Ali	CS	20
102	Sara	IT	22
103	Ahmed	CS	21

- First row `(101, Ali, CS, 20)` is a **tuple**.
- Second row `(102, Sara, IT, 22)` is another **tuple**.

Attribute

- Each column in the table represents one feature or characteristic of that table, and is known as the ***attribute***
- All the attributes collectively will represent one record of a table
- Each attribute has a **unique name** and a **domain** (the set of allowed values).
- Example
  - Attribute Name: Age
  - Age domain = positive integers

Attribute

- For example, in the given Instructor table, ***ID*** represents one of the attributes of the table
- Similarly, ***Name***, ***Dept_Name***, and ***Salary*** are all different attributes of the table
- Using ***ID***, ***Name***, ***Dept_Name***, and ***Salary***, you can access any one complete record

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Relation

- **Each table in the Relational Data Model is known as relation**
- **Since each row represents a relationship among a set of values from each column, so a table is considered as a collection of relationships, as there are more than one rows in the table**
- **As each row is called a tuple with certain number of attributes, let’s say n attributes, hence a single record in the relation is said to be an n-tuple values**

Relation

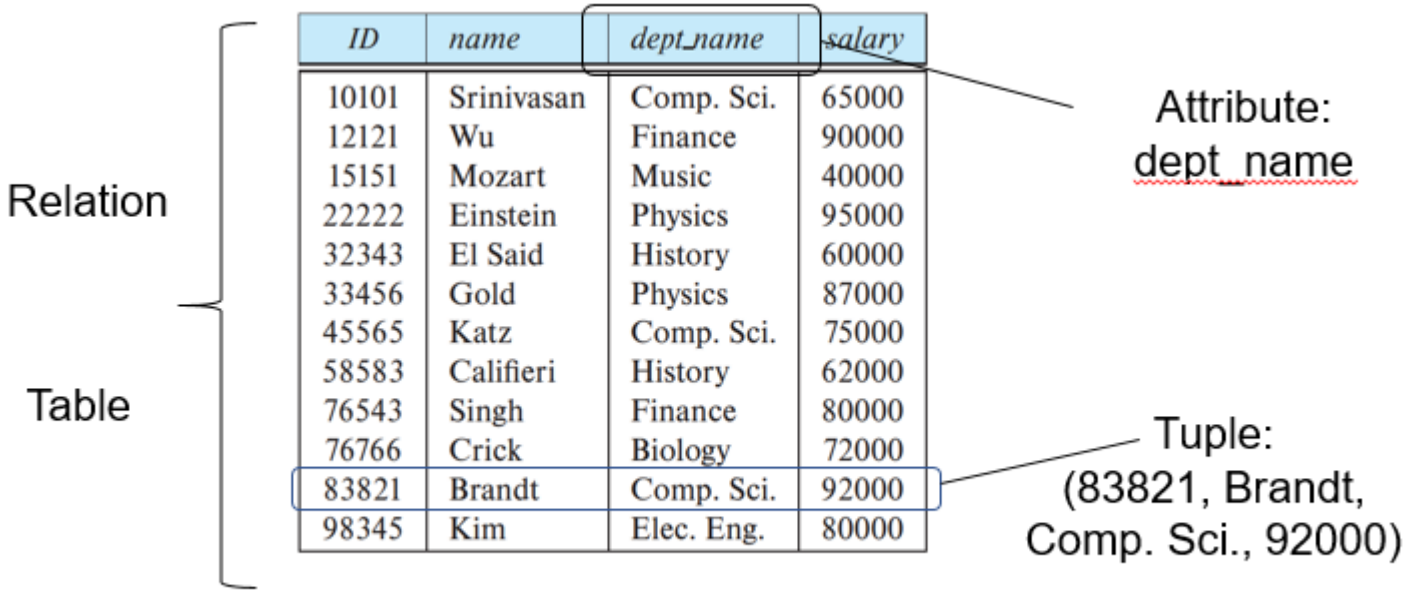
ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Domain

- **Each attribute has a set of permitted values, which are called the domain of that attribute**
- **A domain must be atomic which means they must be singular, indivisible values**
- **The null value signifies that the attribute value is either unknown or does not exist**
- **Null values can cause a number of problems while accessing data, so they should be eliminated, or at least handled when querying the database**
- **Even if the null values are being handled during querying process, key attribute can NEVER have a null value**

Example



ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 2.1 The *instructor* relation.

Example

- Identify some **attributes** in the course relation in the given figure.
- Identify any **tuple** in the relation.
- For each attribute in the relation, are there any attributes that have **unique** values?
- Are there any attributes with **non-unique** values?

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Figure 2.2 The *course* relation.

Example

- Suppose that we add an attribute Phone Number to the instructor relation.
- Is this attribute atomic?
- Can this attribute have a ‘null’ value?

ID	*name*	*dept_name*	*salary*	Phone Number
10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	0300-8877995
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	0092336998956
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	null
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Database Schema

INSTANCES AND SCHEMAS

- ***Instance of the Database:***
  - The collection of information stored in Database at any particular moment in time
- ***Database schema***
  - The overall design of the database
  - Schemas rarely change, as they have already been planned and designed at the Database design stage

INSTANCES AND SCHEMAS

- Schemas are represented as relations, and modeled using Schema Diagrams
- ***Relational Schema:***
  - Specify the name of the relation, as well as its attributes and sometimes indicating the primary keys as underlined attributes
  - When mentioning the primary key, it should always be placed first in the list of attributes
  - For example, Instructor relation will have the following schema:
    - ***Instructor(ID, Name, Dept_Name, Salary)***
  - And Course relation schema will be:
    - ***Course(CourseID, Title, Dept_Name, Credits)***

Comparison

Comparison: Relation vs Tuple vs Attribute

Concept	What it is	Database Representation	Example from STUDENT Table
Relation	A table that stores data about entities or relationships.	Entire table (rows + columns).	`STUDENT (StudentID, Name, Program, Age)`
Tuple	A single row/record in a relation. Represents one instance of the entity.	One row of the table.	`(101, Ali, CS, 20)`
Attribute	A column that represents a property/characteristic of the entity.	One column of the table.	`StudentID`, `Name`, `Program`, `Age`

Keys

Keys

- In the **Relational Data Model**, **keys** are special attributes (or combinations of attributes) used to:
  - **uniquely identify records** and
  - establish **relationships** between tables.
- **There are different types of keys:**
  - **Super key**
  - **Candidate Key**
  - **Primary Key**
  - **Foreign Key**
  - **Surrogate Key**

Super key

- A set of one or more attributes that can uniquely identify a tuple in a relation.
- May contain extra/irrelevant attributes.
- In STUDENT(StudentID, Name, Program, Age)
  - {StudentID} → Super Key
  - {StudentID, Name} → Also a Super Key (but not minimal)

Candidate Key

- A minimal set of super keys are called Candidate keys.
- A relation can have multiple candidate keys.
- Example:
  - {StudentID} is a candidate key.
  - {Name} is not a candidate key (names may repeat).
  - {Email} is another candidate key.

Primary Key

- One or more than one candidate keys chosen by the database designer as the principal means of identifying tuples in a relation is known as the ***primary key.***

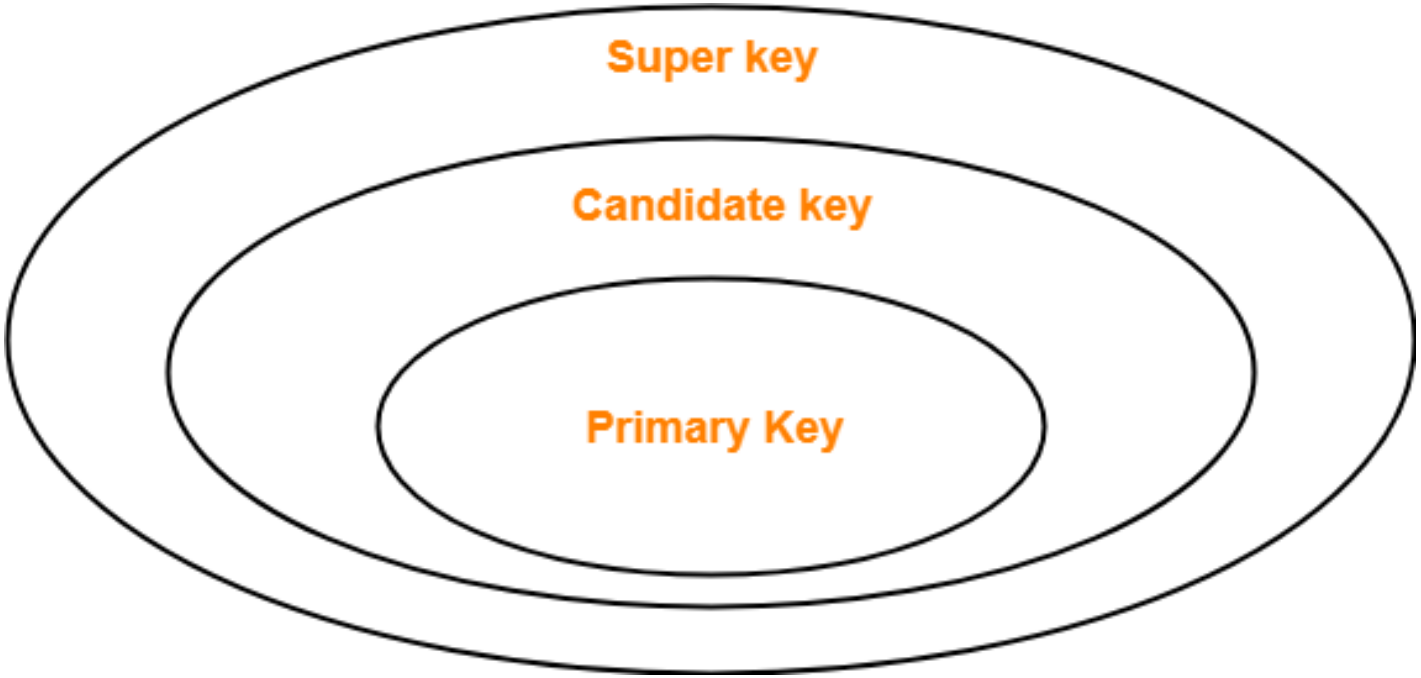


Table: Super Keys vs Candidate Keys vs Primary Key

Super Keys (all possible sets that uniquely identify tuples)	Candidate Keys (minimal superkeys)	Primary Key (chosen candidate key)
{StudentID}	{StudentID}	{StudentID}
{Email}	{Email}	
{StudentID, Email}		
{StudentID, Name}		
{StudentID, Age}		
{Email, Name}		
{Email, Age}		
{StudentID, Email, Name}, {StudentID, Email, Age}, ... (and so on, adding extra attributes)		

Foreign Key

- An attribute in one table that refers to the primary key in another table.
- Enforces Referential Integrity.
- Example:
  - STUDENT(StudentID, Name, Program)
  - ENROLL(StudentID, CourseID)
  - Here, StudentID in ENROLL is a foreign key referencing StudentID in STUDENT.

Surrogate Key

- A surrogate key is an artificial or system-generated key used to uniquely identify a record in a table.
- Unlike natural keys (based on real-world attributes), a surrogate key has no business meaning — it's just an identifier.
- Commonly implemented as an auto-increment integer or a UUID.
- Example
  - 550e8400-e29b-41d4-a716-446655440000

Example - Relational Schema with Keys

Example

The major to-dos when designing our schema are:

1. **Understand business needs**
2. **Identify entities**
3. **Identify properties/fields on those entities**
4. **Define relationships between tables**

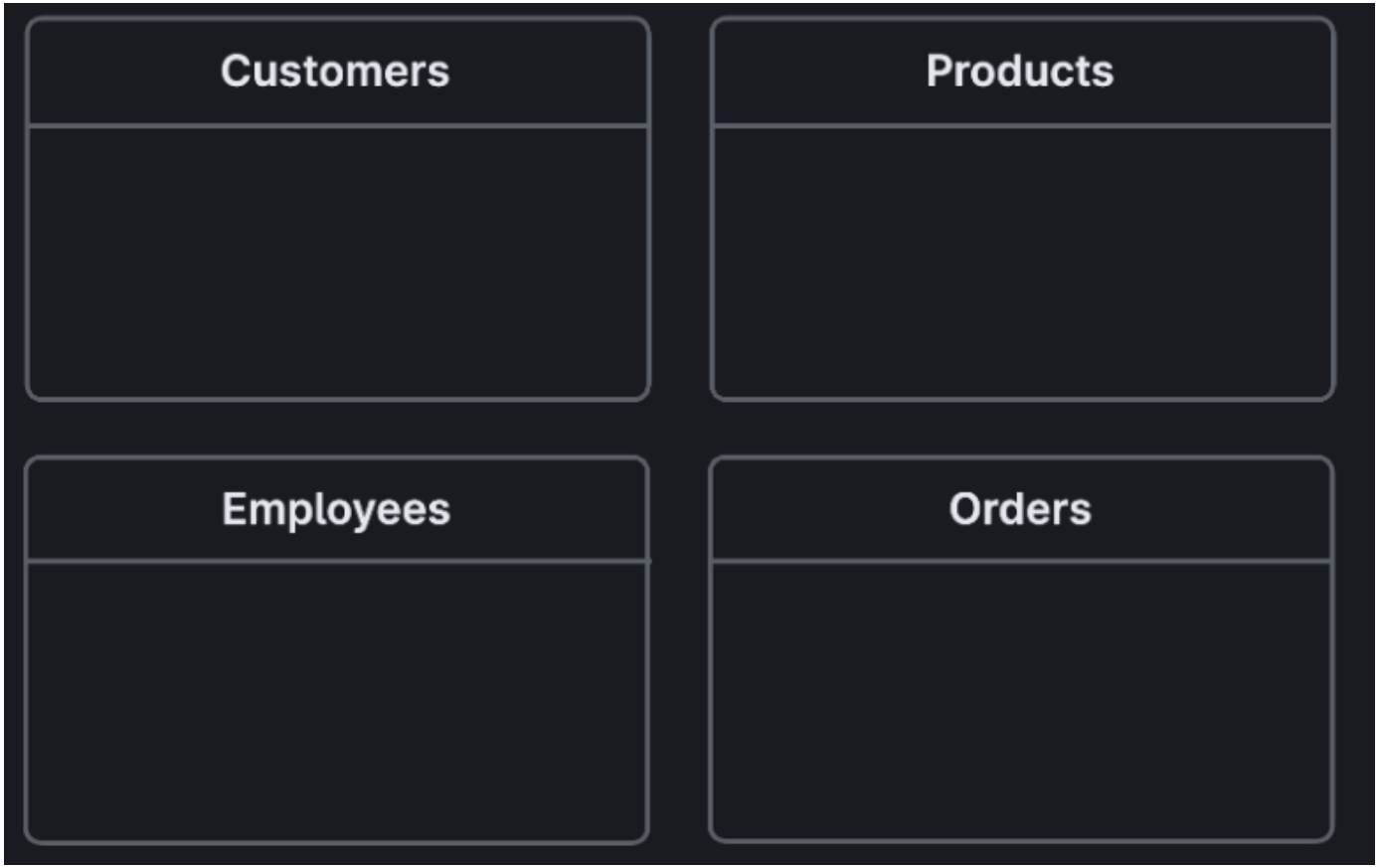
Step 1 – Understanding Requirements

we are working with a retailer that wants to offer an anniversary gift for clients on their first anniversary, we would have to store the date a customer joins.

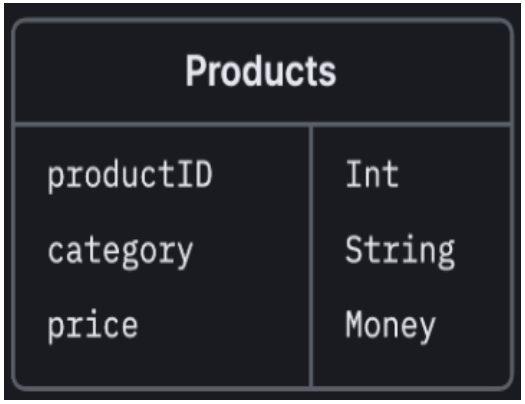
A recap of the requirements for our customers:

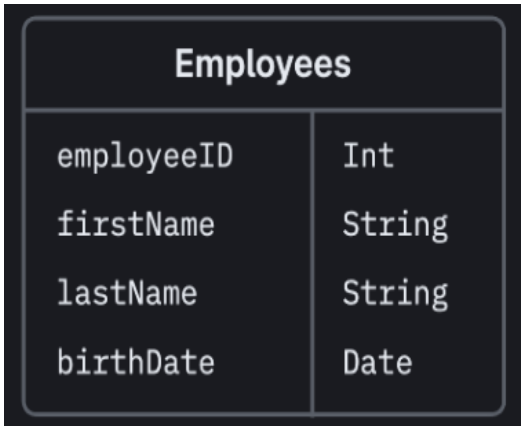
1. **Store customer spending to-date**
2. **Store customer anniversary date of first purchase**
3. **Store customer's age**
4. **Store employee sales total in dollar amount**
5. **Store products and include a category and price property**

Step 2 – Define Entities

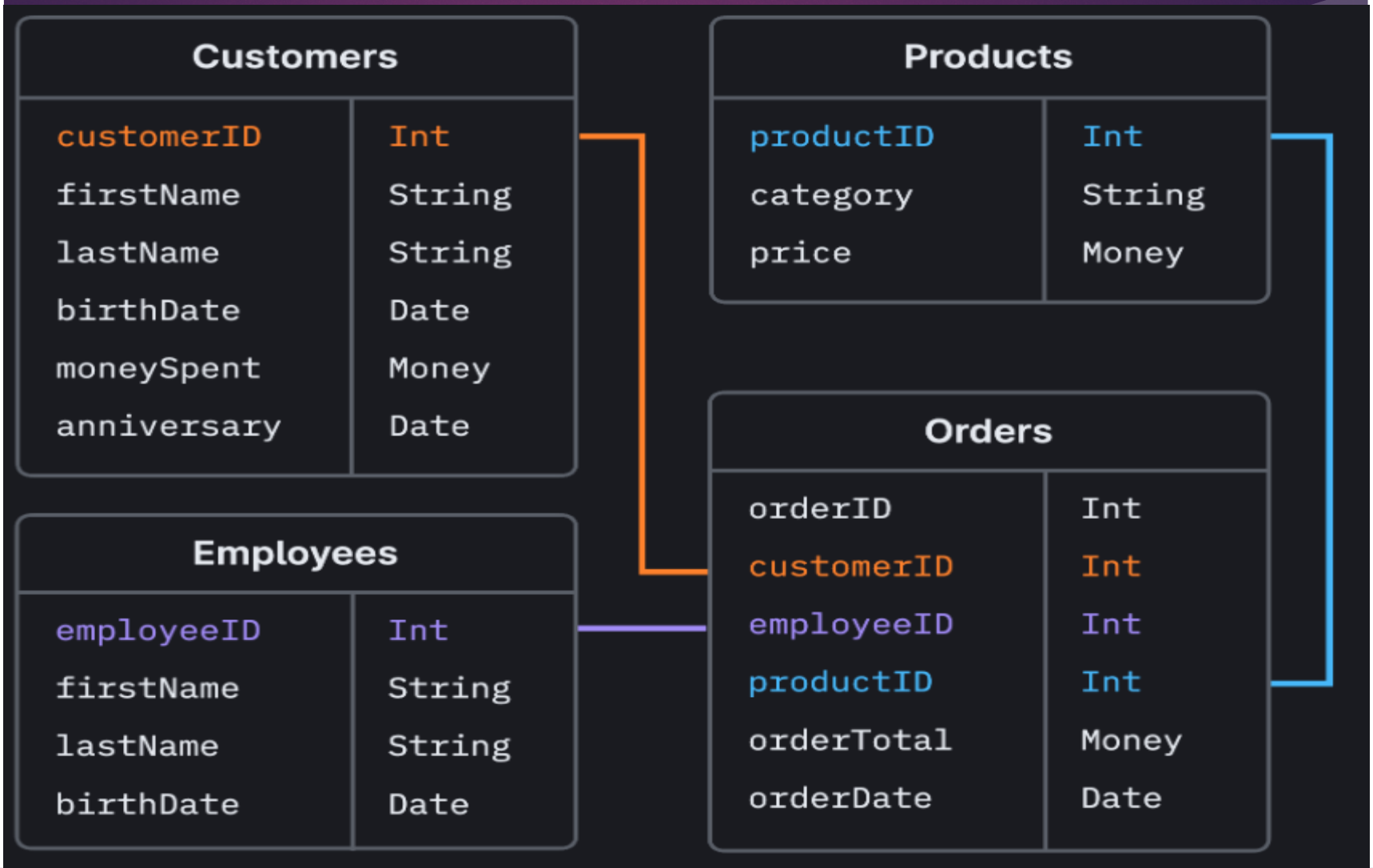


Step 3 – Define Attributes





Step 4 – Define Relationships



THANK YOU